NVIDIA DGX Spark

ARM SIMD 命令の
オセロ思考ルーチンへの活用

NEON / SVE2 によるビットボード高速化技法

C++ サンプルコード + SIMD 命令解説 + API 仕様付き

2026 年 4 月

1. はじめに

オセロ（リバーシ）は 8×8 の盤面上で黒と白の石を交互に置き、相手の石を挟んで裏返すゲームです。コンピュータオセロでは、いかに高速に大量の局面を探索できるかが強さに直結します。

NVIDIA DGX Spark は、GB10 Grace Blackwell Superchip を搭載したデスクトップ AI ワークステーションです。CPU には ARMv9.2 アーキテクチャの Cortex-X925（10 コア）と Cortex-A725（10 コア）を採用しており、NEON（128 ビット固定長 SIMD）と SVE2（128 ビットスケーラブルベクトル拡張）の 2 つの SIMD 命令セットをサポートしています。

本資料では、これらの SIMD 命令がオセロの思考ルーチンの各処理にどのように活用できるかを、図・C++サンプルコード・SIMD 命令解説・API 仕様を交えて詳しく解説します。

2. ビットボードと SIMD レジスタの対応

オセロの盤面は 8×8＝64 マスです。各マスの状態を 1 ビットで表現すれば、盤面全体をちょうど 64 ビット整数 1 つで表せます。黒石の配置を表す 64 ビットと白石の配置を表す 64 ビット、合計 128 ビットで盤面全体を保持するこの手法を「ビットボード」と呼びます。

DGX Spark の NEON/SVE2 レジスタはちょうど **128 ビット幅**です。つまり、黒と白のビットボードを SIMD レジスタ 1 本にぴったり格納できます。

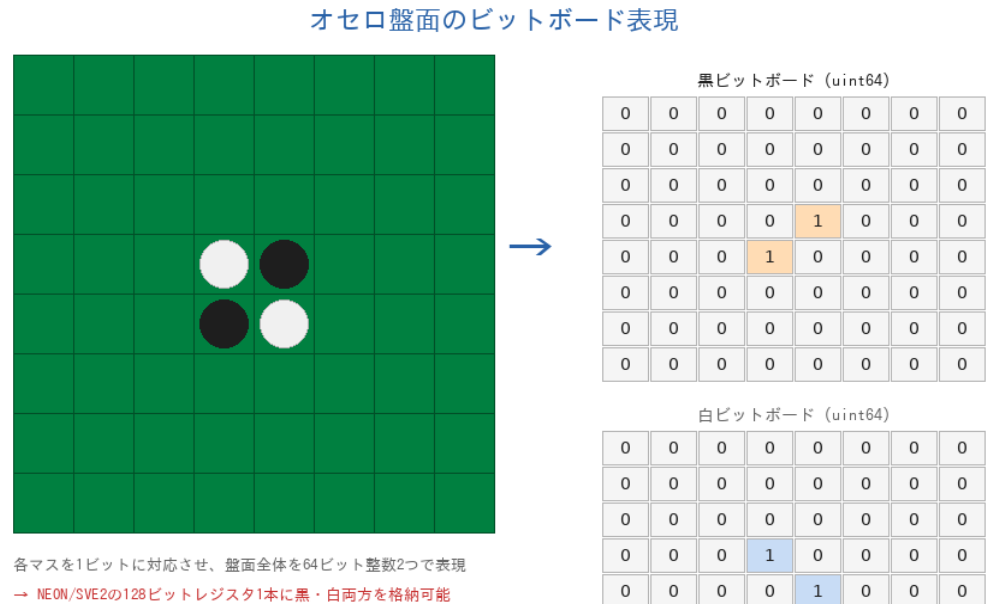


図 1: オセロ盤面のビットボード表現と SIMD レジスタへの格納

コード例: ビットボードの NEON レジスタ操作

以下のコードでは、黒白 2 つの 64 ビットボードを NEON の 128 ビットレジスタ（uint64x2_t）に格納し、ロード・ストアする基本操作を示します。

```
#include <arm_neon.h>

struct Board {
    uint64_t black;   // 黒石ビットボード
    uint64_t white;   // 白石ビットボード
};

/*
 * load_board(b)
 *   概要 : Board 構造体の黒白ビットボードを
 *          NEON の 128bit レジスタ 1 本にまとめてロードする
 *   b    : const Board& — ロード元の盤面構造体
 *            b.black → レーン 0 (下位 64bit)
 *            b.white → レーン 1 (上位 64bit)
 *   戻値 : uint64x2_t  — 黒白を格納した 128bitSIMD レジスタ
 */
inline uint64x2_t load_board(const Board& b) {
```

```
    uint64_t data[2] = { b.black, b.white };
    /*
     * vld1q_u64(ptr)
     *   ptr : const uint64_t* — ロード元アドレス
     *         ptr[0](黒) → レーン 0, ptr[1](白) → レーン 1
     *   戻値: uint64x2_t     — 128bitSIMD レジスタ
     */
    return vld1q_u64(data);
}

/*
 * store_board(v)
 *   概要 : NEON の 128bit レジスタから Board 構造体へ書き戻す
 *   v    : uint64x2_t  — 書き戻す SIMD レジスタ
 *            レーン 0 (下位 64bit) → b.black
 *            レーン 1 (上位 64bit) → b.white
 *   戻値 : Board — 復元された盤面構造体
 */
inline Board store_board(uint64x2_t v) {
    Board b;
    uint64_t data[2];
    /*
     * vst1q_u64(ptr, val)
     *   ptr : uint64_t*   — 書き込み先アドレス
     *   val : uint64x2_t  — ストアする SIMD レジスタ
     *         レーン 0 → ptr[0](黒), レーン 1 → ptr[1](白)
     *   戻値: void
     */
    vst1q_u64(data, v);
    b.black = data[0];
    b.white = data[1];
    return b;
}

/*
 * get_empty(b)
 *   概要 : 盤面上の空きマス(黒でも白でもないマス)を
 *          ビットマップとして取得する
 *   b    : const Board& — 判定対象の盤面構造体
 *   戻値 : uint64_t     — 空きマスのビットマップ
 *            1 のビット位置 = 空きマス
 */
inline uint64_t get_empty(const Board& b) {
    uint64x2_t bw = load_board(b);
    /*
     * vorrq_u64(a, b)
     *   a    : uint64x2_t — 第 1 オペランド(黒ビットボード)
     *   b    : uint64x2_t — 第 2 オペランド(白ビットボード)
     *   戻値 : uint64x2_t — a | b の各レーンビット OR 結果
     */
    uint64x2_t occupied = vorrq_u64(
        vreinterpretq_u64_u32(
            /*
             * vmvnq_u32(a)
             *   a    : uint32x4_t — 入力ベクトル
             *   戻値 : uint32x4_t — ~a 全ビット NOT 結果
             *   ※ 黒|白 を反転 → 空きマスのビットが 1 になる
             */
            vmvnq_u32(vreinterpretq_u32_u64(bw))),
        vreinterpretq_u64_u32(
            vmvnq_u32(vreinterpretq_u32_u64(bw))));
    return vgetq_lane_u64(occupied, 0);
```

```
}
```

リスト 1: NEON intrinsic を用いたビットボードの 128 ビットレジスタ操作

◆ SIMD 命令 詳細解説

[1] ― NEON LD1 命令。メモリから 128 ビット(64bit×2 要素)を 1 サイクルで 1 本の SIMD レジスタへロード。黒石(data[0])がレーン 0、白石(data[1])がレーン 1 に格納されます。

[2] ― NEON ST1 命令。SIMD レジスタの 128 ビットをメモリへ 1 命令で書き戻します。ロードの逆操作です。

[3] ― NEON ORR 命令(Quadword/64bit lanes)。2 本の 64bit レーンを同時にビット OR。黒∪白のビットマップを 1 命令で求めます。

[4] ― NEON MVN 命令(Quadword)。全ビットをビット反転(NOT)。ORR 結果に NOT をかけることで「空きマス」ビットマップが 1 命令で完成します。

◆ API 仕様 / 引数説明

関数名 (intrinsic)	戻り値型	主な引数	引数・動作の説明
vld1q_u64	uint64x2_t	const uint64_t *ptr	ptr が指すメモリから 64bit×2 要素(128bit)を連続ロード。ptr[0]→レーン 0、ptr[1]→レーン 1 に格納。アライメント不要。
vst1q_u64	void	uint64_t *ptr, uint64x2_t val	val の 128bit を ptr へ連続ストア。レーン 0→ptr[0]、レーン 1→ptr[1]。vld1q_u64 の逆操作。
vorrq_u64	uint64x2_t	uint64x2_t a, uint64x2_t b	a、b の各対応レーンをビット OR。黒石∪白石の論理和を 2 レーン同時に計算する。
vmvnq_u32	uint32x4_t	uint32x4_t a	a の全ビットをビット反転(NOT)。32bit レーン×4 として処理するが実質 128bit 全反転。空きマスの NOT に使用。
vget_low_u64	uint64_t	uint64x2_t a	128bit ベクトルのレーン 0(下位 64bit)をスカラーと取得。
vget_high_u64	uint64_t	uint64x2_t a	128bit ベクトルのレーン 1(上位 64bit)をスカラーと取得。

3. 合法手生成の 8 方向同時探索

オセロで石を置けるかどうかを判定するには、ある位置から 8 方向（上下左右＋斜め 4 方向）に対して「相手の石が連続し、その先に自分の石がある」かを確認する必要があります。

3.1 スカラー処理の問題点

従来のスカラーコードでは、8 方向それぞれについてループを回す必要がありました。1 方向あたり最大 6 回のシフト＋比較が必要なため、合法手判定だけで多数の命令を消費します。

3.2 SIMD による並列化

SVE2 の 128 ビットレジスタを 8 つの 16 ビットレーンに分割し、各レーンに 1 方向の探索状態を割り当てます。1 つの SIMD シフト命令で 8 方向すべてを同時にシフトし、1 つの AND 命令で 8 方向すべての挟み判定を同時に行えます。

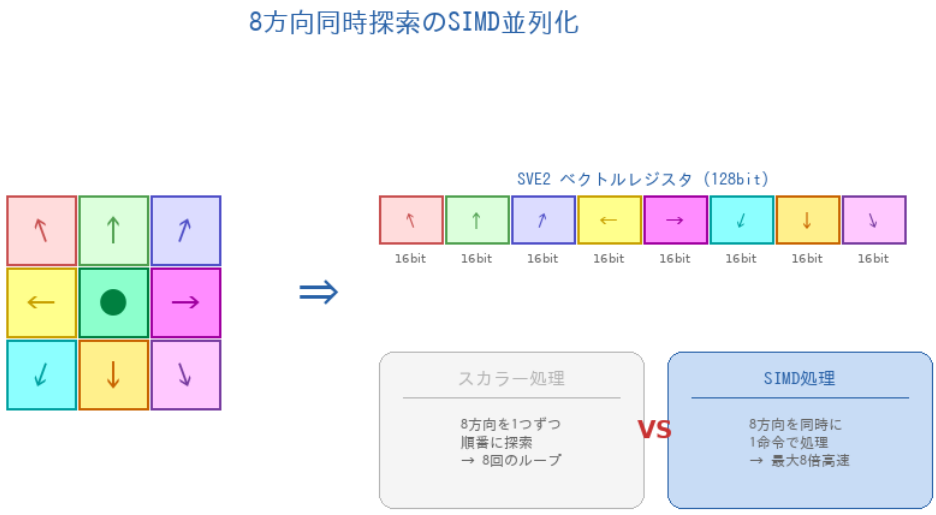


図 2: 8 方向探索の SIMD 並列化の概念

3.3 SIMD の「レーン」とは

SIMD レジスタは、1 本の物理レジスタ内を等幅の区画に分割して使います。この区画 1 つ 1 つを「レーン」と呼びます。DGX Spark の 128 ビットレジスタの場合、64 ビット×2 レーン、32 ビット×4 レーン、16 ビット×8 レーン、8 ビット×16 レーンのいずれかに分割できます。

1 つの SIMD 命令を発行すると、全レーンで同一の演算が同時に実行されます。例えば 128 ビットレジスタを 64 ビット×2 レーンに分割すれば、1 命令のシフトで「右方向のシフト」と「左方向のシフト」を同時に処理できます。レーン数を増やすほど並列度が上がりますが、1 レーンあたりのビット幅は狭くなります。



図 2-補足 1: SIMD レーンの概念 ― 128 ビットレジスタの分割パターン

リスト 2 では 64 ビット×2 レーンを使い、1 回の SVE2 演算で 2 方向を同時に処理しています。8 方向を 2 方向ずつ 4 ペアに分けることで、ループ回数を半減させています。

3.4 述語制御の SIMD 命令とは

SVE2 の命令には、ほぼすべてに「述語レジスタ」（プレディケートレジスタ、型名 svbool_t）を第 1 引数として渡します。述語レジスタは各レーンに対して TRUE（処理する）または FALSE（スキップする）を指定するビットマスクです。

述語制御の最大の利点は、**条件分岐（if 文）を完全に排除できること**です。スカラー処理では「このレーンを処理するか？」を毎回 if 文で判定しますが、SVE2 では述語が TRUE のレーンだけが自動的に演算され、FALSE のレーンはハードウェアレベルでスキップされます。分岐予測ミスによるパイプラインストールが一切発生しません。



図 2-補足 2: 述語(プレディケート)制御の仕組みとリスト 2 での使われ方

リスト 2 では svptrue_b64() で全レーンを TRUE にした述語を使っています。これは全レーンを対象に演算したいからです。一方、第 5 章の述語フィルタリングでは svwhilelt や svcmpne で条件に応じた述語を動的に生成し、合法手レーンだけを選択的に処理します。

SVE2 命令の**_x サフィックス**（svlsr_x、svand_x 等）は、述語が FALSE のレーンの結果が「未定義」でよいことを示します。ハードウェアは未定義レーンの演算を完全にスキップできるため、**_z（ゼロ埋め）や _m（マージ）よりも高速**に実行されます。リスト 2 では全レーン TRUE ですが、将来の拡張で部分的に FALSE になる場合も _x が最適です。

3.5 なぜシフト＋AND で合法手が分かるのか

ここまで SVE2 命令の並列化技法を解説しましたが、そもそも「自石をシフトして相手石と AND を取る」操作を繰り返すだけで、なぜ石を置ける場所（合法手）が分かるのでしょうか。この節ではアルゴリズムの原理を具体例で説明します。

オセロで石を置けるマスの条件は、**①着手位置のすぐ隣から相手石が 1 個以上連続しており、②その先に自石がある**ことです。つまり「相手石を自石で挟んでいる」方向が 1 つでもあれば、そこは合法手です。

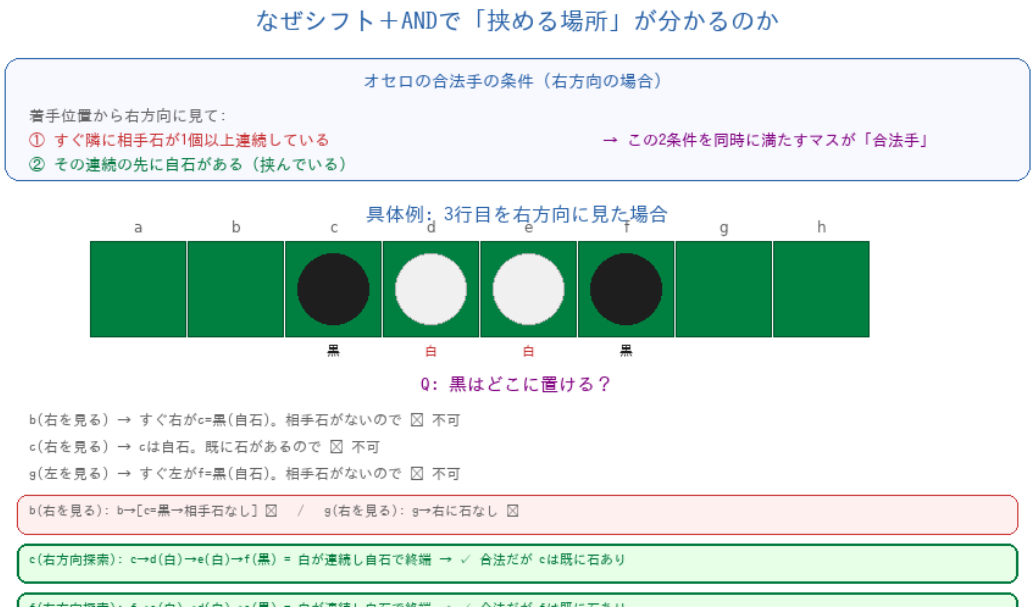


図 2-補足 3: オセロの合法手の条件（右方向の例）

ビット演算では、この条件判定を次の 3 つの操作に分解します。

原理①: シフト＝「隣のマスに進む」操作。ビットを 1 つ右シフトすることは、盤面上で 1 マス右に進むことと同じです。下方向なら 8 ビットシフト（1 行＝8 マス）、斜め右下なら 9 ビットシフトに対応します。

原理②: AND(opp)＝「進んだ先に相手石があるか」の判定。シフト後のビットと相手石マスクの AND を取ると、進んだ先に相手石があれば 1、なければ 0 になります。1 なら「挟み候補」として蓄積し、シフトを継続します。

原理③: 最後の AND(my)＝「自石で挟んでいるか」の確認。相手石の連続が途切れた先のビットと自石マスクの AND を取ります。自石があれば挟み成立＝合法手確定。自石がなければ挟めないので不合法です。

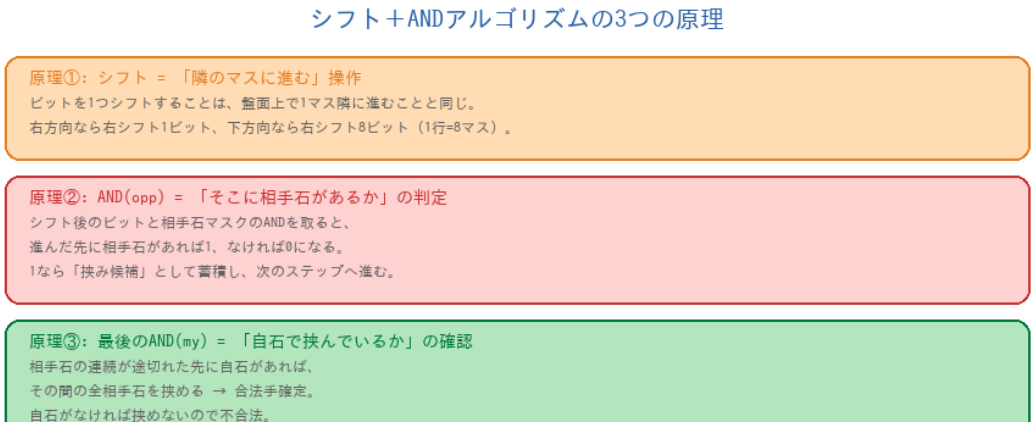


図 2-補足 4: シフト +AND アルゴリズムの 3 つの原理

3.5.1 成功例: 合法手が見つかる場合

具体的なビット操作を追いかけてみましょう。下図の例では、自石が b 列と h 列にあり、相手石が e,f,g 列にある盤面を考えます。d 列（空きマス）に黒を置けるか、右方向を探索します。



図 2-補足 5: ビット演算で合法手を発見する過程（右方向、成功例）

d 地点にカーソル（仮の石）を置き、右に 1 ビットずつシフトしながら AND(opp)を繰り返すと、e→f→g と相手石が連続することが確認できます。次のシフトで h 地点に達し、AND(my)が 1 になります。**これは「d に石を置くと e,f,g の白石を h の黒石で挟める」ことを意味し、d が合法手として確定します。**

3.5.2 失敗例: 合法手にならない場合

一方、相手石の先に自石がない場合はどうなるでしょうか。



図 2-補足 6: 自石に到達しない場合（合法手にならない例）

d からシフトして e(白)→f(白)と相手石の連続を確認しますが、次の g 地点には AND(opp)も AND(my)も 0 です。**相手石の連続の先に自石がないため挟めず、d は合法手ではありません。**プログラム上では svcmpne_u64 の結果が 0（FALSE）となり、legal ビットに d は追加されません。

このように、「シフト（移動）→AND(opp)（相手石確認）→繰り返し→AND(my)（自石到達確認）」の一連のビット操作は、**オセロの「相手石を自石で挟む」というルールをビット演算に直接翻訳したもの**です。この操作を SVE2 で全方向同時に実行することで、分岐なしに高速な合法手判定が実現されます。

3.6 なぜ全自石の一斉シフトですべての合法手が求まるのか

3.5 節では 1 つの起点からの探索を例示しましたが、リスト 2 のコードでは **svdup_u64(my)** でビットボード全体をシフトしています。これは 64 マスの全自石を同時に 1 マス進める操作です。この「全自石一斉シフト」でなぜ盤面上の**すべての**合法手が見つかるのかを説明します。

3.6.1 1 マス探索と全自石一斉シフトの違い

素朴な方法では、64 マスそれぞれについて 8 方向を調べるため最悪 512 回のループが必要です。一方、ビットボードによる全自石一斉シフトでは、**1 回のシフト命令で全 64 ビット（＝全マス）が同時に移動する**ため、8 方向×6 ステップ＝48 回の演算ですべての合法手が同時に確定します。

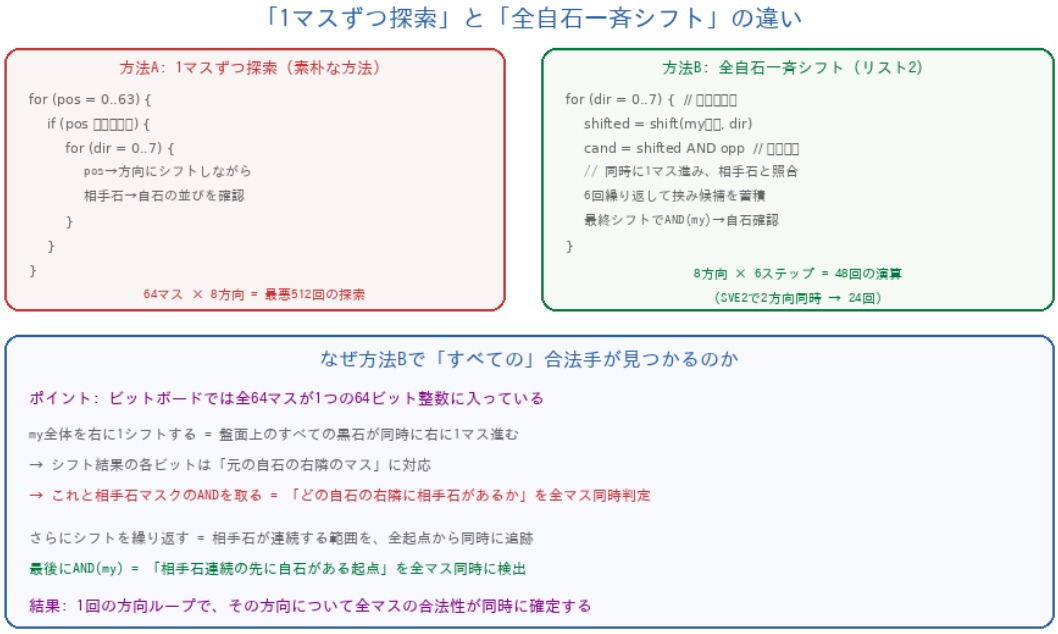


図 2-補足 7: 1 マスずつ探索（方法 A）と全自石一斉シフト（方法 B）の対比

ビットボードの核心は、**64 マスの状態が 64 ビット整数の各ビットに対応している**ことです。my 全体を右に 1 ビットシフトすると、a 列の自石は b 列に、b 列の自石は c 列に……と、盤面上のすべての黒石が同時に右に 1 マス移動したのと等価になります。

このシフト結果と相手石マスク(opp)の AND を取ると、**「どの自石の右隣に相手石があるか」が全マス分、一度に判定**されます。さらにシフト＋AND を繰り返すことで、相手石が何マス連続していても追跡でき、最後に空きマスまたは自石に到達したかを確認すれば、すべての合法手が確定します。

3.6.2 具体例: 2 つの自石から同時に探索

下図では、a 列と e 列にある 2 つの黒石が右方向に同時にシフトされる様子を示します。1

回のシフト＋AND 操作で、2 つの起点からの探索が並列に進みます。



図 2-補足 8: 2 つの自石が同時にシフトされる具体例（右方向）

この例では、a 列の黒石は右隣 b 列が空きマスのため step1 で AND(opp)=0 となり脱落します。一方、e 列の黒石は右隣 f 列に白石があるため探索が継続し、f→g（白石連続）→h（空き）へと進みます。**自石が何個あっても同じ回数の演算で完了する点がビットボードの威力です。**

3.6.3 正確なアルゴリズムの全体像

リスト 2 の概念コードを正確に記述すると、各方向(dir)について 3 つのステップで構成されます。



図 2-補足 9: 正確な合法手生成アルゴリズムの全体像

Step 1: 全自石を 1 マス進め、すぐ隣に相手石があるマスを検出します。**Step 2**: 発見した相手石列をさらに先へ拡張し（最大 5 回繰り返し）、何マス連続する相手石でも追跡できるようにします。**Step 3**: 相手石連続の先にある空きマスを検出し、それを合法手として確定します。

ここで重要なのは、探索の方向です。コード上では「自石→dir 方向に相手石列→その先の空きマス」を検出していますが、これは「空きマスから**逆方向**に相手石列→自石」の存在と同値です。つまり、**「石を置いたら相手石を挟める空きマス」を全マス同時に見つけている**のです。

8 方向すべての legal を OR で統合すれば、盤面上のすべての合法手のビットマスクが完成します。1 つでも方向で挟めるマスは合法手として検出されます。

3.7 リスト 2 のコードの正確な読み方（訂正と補足）

ここまでの 3.5 節・3.6 節の説明では、リスト 2 のコードが「自石を各方向にシフトして相手石と AND を取り、最後に自石に到達したら合法手」と解説しました。しかし、この説明には重要な不正確さがありました。

ご指摘の通り、リスト 2 のコードをそのまま読むと「ある位置に石を置いたときに合法手かどうかをチェックする処理」に見えます。盤面全体のすべての合法手を求めるには、**空きマス 1 つ 1 つに仮置きする処理が追加で必要なのではないか**、という疑問は正当です。

実は、正しいビットボード合法手生成アルゴリズムでは、空きマスへの仮置きループは不要です。その理由は、**探索の方向を逆転させている**からです。

3.7.1 正しいアルゴリズム: 自石→相手石列→空きマス

直感的には「空きマスに石を置いて、そこから相手石列を辿り、自石にぶつかるか確認」したくなります。しかしビットボードでは逆方向に考えます。

Step 1: 全自石を 1 マス dir 方向にシフトし、相手石マスクと AND を取る。これで「自石のすぐ隣に相手石があるマス」が全自石分、同時に検出されます。

Step 2: 検出した候補をさらに dir 方向にシフトし、相手石と AND を取る。これを最大 5 回繰り返して、相手石の連続列を拡張します。

Step 3: 最終的な候補をもう 1 回 dir 方向にシフトし、今度は**空きマスマスク(empty = ~(my|opp))と AND** を取る。結果が 1 のビット位置が合法手です。

つまり最終ステップで照合するのは**自石(my)ではなく空きマス(empty)**です。「自石→相手石列→空きマス」の順に探索し、空きマスを発見する方式なので、空きマスに 1 つ 1 つ仮置きするループは不要です。

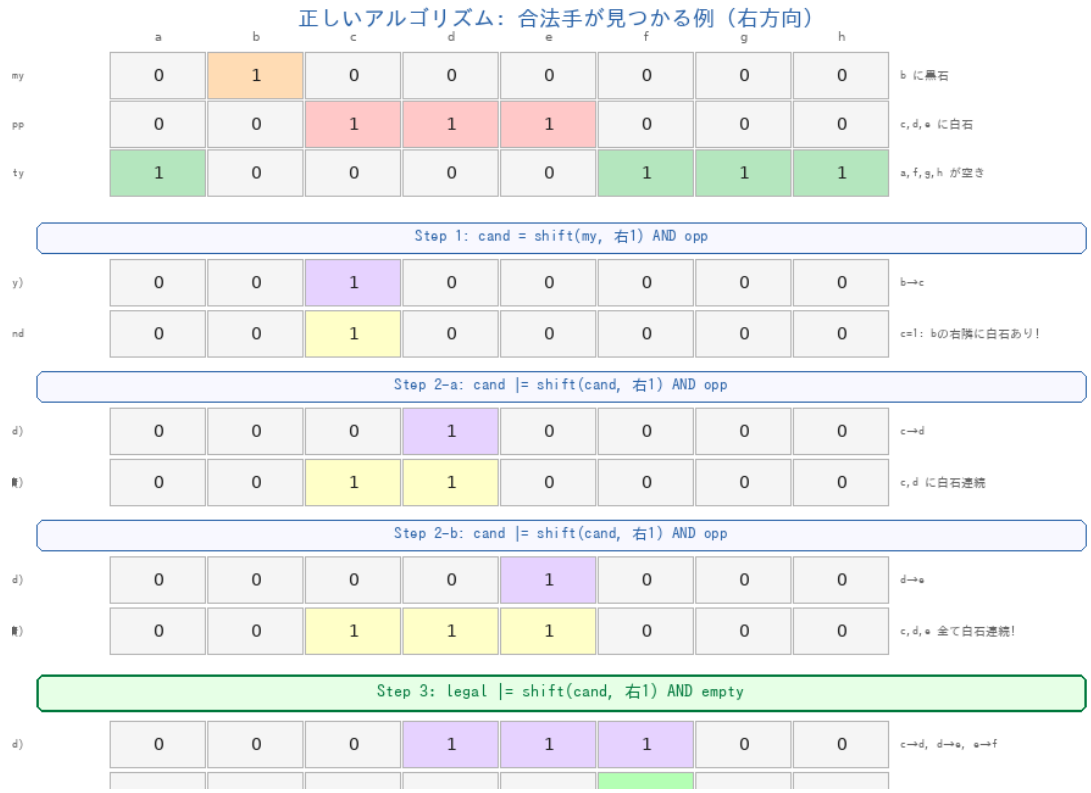


図 2-補足 10: 正しいアルゴリズムの動作例 ― 自石(b)→白石列(c,d,e)→空きマス(f)を検出

上図の例では、b の黒石から右方向にシフトを繰り返すと、c→d→e（白石連続）を追跡できます。最終シフトで f 地点に到達し、AND(empty)が 1 になります。**これは f に黒を置けば c,d,e の白石を挟めることを意味します。**

3.7.2 リスト 2 のコードとの対応

リスト 2 の概念コードでは最終ステップで svcmpne_u64(..., my, 0) と書いており、AND(my)で自石に到達するかを確認しているように見えます。これは教育目的の簡略化であり、正しい実装では Step 3 で **AND(empty) を使い、相手石列の先にある空きマスを直接検出**します。

なお、AND(my)で自石到達を確認する方式は「flip 処理（リスト 3）」で使われる手法です。flip 処理ではすでに着手位置が決まっており、そこから挟まれる相手石を特定するために AND(my)が必要です。合法手生成と flip 処理は似ていますが、最終ステップが異なる点に注意してください。



図 2-補足 11: 正しい合法手生成アルゴリズムのまとめ

この「自石側からの押し出し探索」により、全自石から全方向の合法手が空きマスへの仮置きなしに一括検出できます。8 方向すべての legal を OR で統合すれば、盤面上の全合法手のビットマスクが完成します。

コード例: SVE2 による合法手生成

各方向のシフト量と端マスクをベクトルにロードし、SVE2 命令で並列にシフト＋AND 処理を行います。

```
#include <arm_sve.h>

static const int      SHIFT_AMOUNTS[8] =
    { 1,-1,8,-8,9,-9,7,-7 };
static const uint64_t EDGE_MASK[8] = {
    0xFEFEFEFEFEFEFEFEULL, 0x7F7F7F7F7F7F7F7FULL,
    0xFFFFFFFFFFFFFF00ULL, 0x00FFFFFFFFFFFFFFULL,
    0xFEFEFEFEFEFEFE00ULL, 0x007F7F7F7F7F7F7FULL,
    0x7F7F7F7F7F7F7F00ULL, 0x00FEFEFEFEFEFEFEull,
};

/*
 * generate_legal_moves_sve2(my, opp)
 *   概要 : SVE2 命令で 8 方向を並列処理し、自石が置ける
 *          すべてのマスをビットマスクとして返す
 *   my   : uint64_t — 自石のビットボード
 *            1 のビット位置 = 自石があるマス
 *   opp  : uint64_t — 相手石のビットボード
 *            1 のビット位置 = 相手石があるマス
 *   戻値 : uint64_t — 合法手のビットマスク
 *            1 のビット位置 = 石を置けるマス
 */
uint64_t generate_legal_moves_sve2(
        uint64_t my, uint64_t opp) {

    /*
     * svptrue_b64()
     *   戻値: svbool_t — 全 64bit レーンを TRUE にした述語
     */
    svbool_t pg = svptrue_b64();
```

```
    uint64_t legal = 0;

    for (int step = 0; step < 6; step++) {
        for (int d = 0; d < 8; d += 2) {
            svuint64_t v_my  = svdup_u64(my);
            svuint64_t v_opp = svdup_u64(opp);
            svuint64_t v_mask =
                svld1_u64(pg, &EDGE_MASK[d]);

            int s0 = SHIFT_AMOUNTS[d];
            svuint64_t shifted =
                (s0>0) ? svlsr_x(pg,v_my,s0)
                       : svlsl_x(pg,v_my,(uint64_t)(-s0));

            svuint64_t cand = svand_x(pg,shifted,v_opp);
            cand = svand_x(pg,cand,v_mask);

            svbool_t found = svcmpne_u64(pg,
                svand_x(pg,
                    svlsr_x(pg,cand,
                        (uint64_t)(s0>0?s0:-s0)),
                    v_my),
                0);
            legal |= svlastb_u64(found, cand);
        }
    }
    return legal;
}
```

リスト 2: SVE2 intrinsic を用いた合法手生成（概念コード）

リスト 2 の動作解説

以下の図で generate_legal_moves_sve2 関数の処理フローをステップごとに解説します。

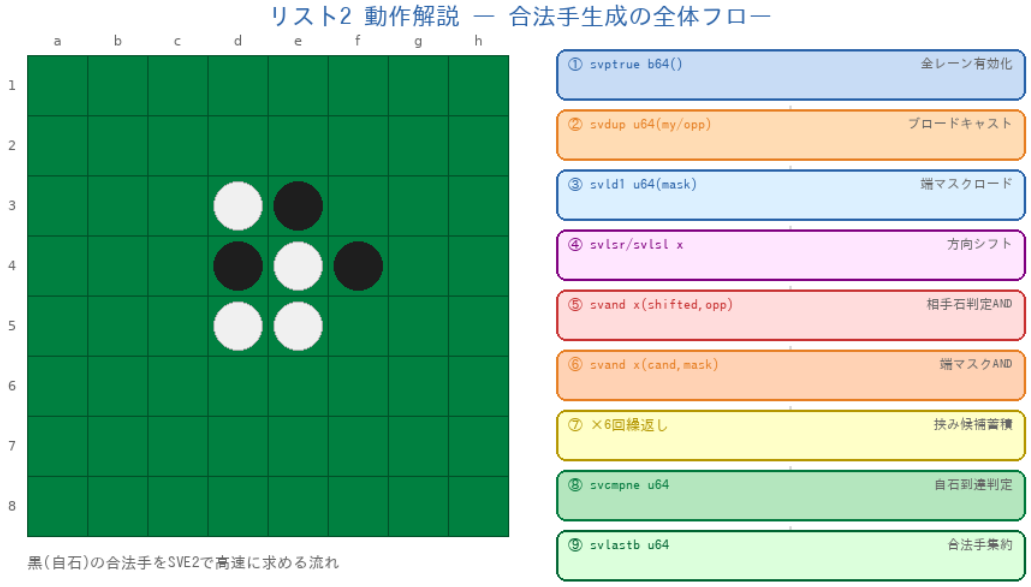


図 2a: 合法手生成の全体フロー（9 ステップの処理順序）

まず svptrue_b64 で全レーンを有効化した述語を作り、svdup_u64 で自石・相手石のビットボードを全レーンにブロードキャストします。次に svld1_u64 で端マスク配列から 2 方

向分のマスクを同時にロードします。

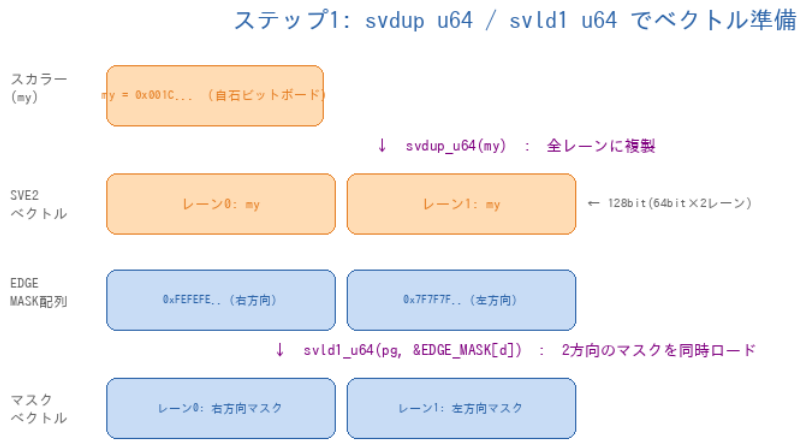


図 2b: ステップ 1 — svdup_u64 / svld1_u64 でベクトルにデータを準備

128 ビットレジスタは 64 ビット×2 レーン構成のため、1 回の SVE2 演算で 2 方向を同時処理できます。4 回のペア処理（d=0,2,4,6）で全 8 方向をカバーします。



図 2c: ステップ 2 — 1 方向のシフト → AND → 挟み判定の連鎖

svlsr_x で自石ビットを方向に沿ってシフトし、svand_x で相手石との交差を判定します。さらに端マスクとの AND で盤外への回り込みを防止します。このシフト＋AND を最大 6 回繰り返し、相手石の連続範囲を特定した後、svcmpne_u64 で自石に到達したかを判定して合法手を確定します。



図 2d: ステップ 3 ― 2 レーン並列処理と合法手集約

4 ペア（右+左、下+上、右下+左上、右上+左下）の処理が完了したら、各ペアの結果を svlastb_u64 でスカラーに集約し、ビット OR で legal 変数に統合します。スカラー処理の 48 回のループが、SVE2 では 24 回の SIMD 演算に削減されます。

◆ SIMD 命令 詳細解説

[1] ― 全 64bit レーンを TRUE にした述語レジスタを返します。以降の全演算はこの述語が有効なレーン全体を対象とします。

[2] ― スカラー値を全レーンに複製(ブロードキャスト)したベクトルを生成。my/opp の値を 2 レーン同時に保持します。

[3] ― 述語 pg が有効なレーンのみメモリからロード。端マスク配列の 2 要素を 1 命令でベクトルに取り込みます。

[4] ― 論理右シフト(Logical Shift Right)。pg が有効な全レーンを同一シフト量で一斉にシフト。_x は非アクティブレーンが未定義でよいことを示し最速実行されます。

[5] ― 論理左シフト(Logical Shift Left)。負方向(右→左)の方向探索に使用します。

[6] ― ビット AND。1 回目は相手石マスクとの AND で「シフト先が相手石か」を判定。2 回目は端マスクとの AND で盤外への回り込みを防止します。両者とも全レーン同時処理です。

[7] ― 各レーンを imm と比較し、≠なら述語 TRUE を生成。「自石に到達したか」を条件分岐なしに全レーン同時判定します。

[8] ― 述語 pg が最後に TRUE となったレーンの値をスカラーで返します。合法手ビットの集約に使用します。

◆ API 仕様 / 引数説明

関数名 (intrinsic)	戻り値型	主な引数	引数・動作の説明
svptrue_b64	svbool_t	(引数なし)	全 64bit レーンを TRUE(1)にした述語レジスタを返 演算対象を全レーンとする最もシンプルな述語。
svdup_u64	svuint64_t	uint64_t imm / scalar	スカラー値を SVE2 ベクトルの全レーンに複製(ブロ ドキャスト)したベクトルを返す。
svld1_u64	svuint64_t	svbool_t pg, const uint64_t *ptr	pg が TRUE のレーンのみ ptr からロード。FALSE ンは 0 で埋める。境界外アクセスを防ぐ述語付きロ ド。
svlsr_x	svuint64_t	svbool t pg, svuint64 t op, uint64_t shift	pg が有効なレーンを shift ビット分論理右シフト。_ inactive レーンが未定義=最高速。正方向(右)探索に 応。
svlsl_x	svuint64_t	svbool t pg, svuint64_t op, uint64_t shift	pg が有効なレーンを shift ビット分論理左シフト。 向(左)探索に対応。svlsr_x と対をなす命令。
svand_x	svuint64_t	svbool_t pg, svuint64_t a, svuint64 t b	pg が有効なレーンの a&b をビット AND。相手石判 端マスク適用の両方に使用する汎用 AND 命令。
svcmpne_u64	svbool_t	svbool_t pg, svuint64 t a, svuint64 t b	pg が有効な各レーンで a≠b を比較し結果を述語と 返す。0 との比較で「ビットが立っているか」を判
svlastb_u64	uint64_t	svbool_t pg, svuint64_t v	pg が最後に TRUE のレーンの値をスカラーで返す。 法手ビットをスカラーに集約するために使用。

4. 石の反転処理（フリップ）

石を置いた後、挟まれた相手の石を裏返す「フリップ処理」は、オセロ思考ルーチンで最も頻繁に呼ばれる関数です。探索中に何百万回も実行されるため、1 命令の差が全体の性能に大きく影響します。

4.1 シフト＋AND の連鎖

フリップ処理の基本アルゴリズムは、自石の位置から特定方向にビットシフトし、相手石マスクと AND を取ることで「相手石が連続する範囲」を特定するものです。これを最大 6 回繰り返します。

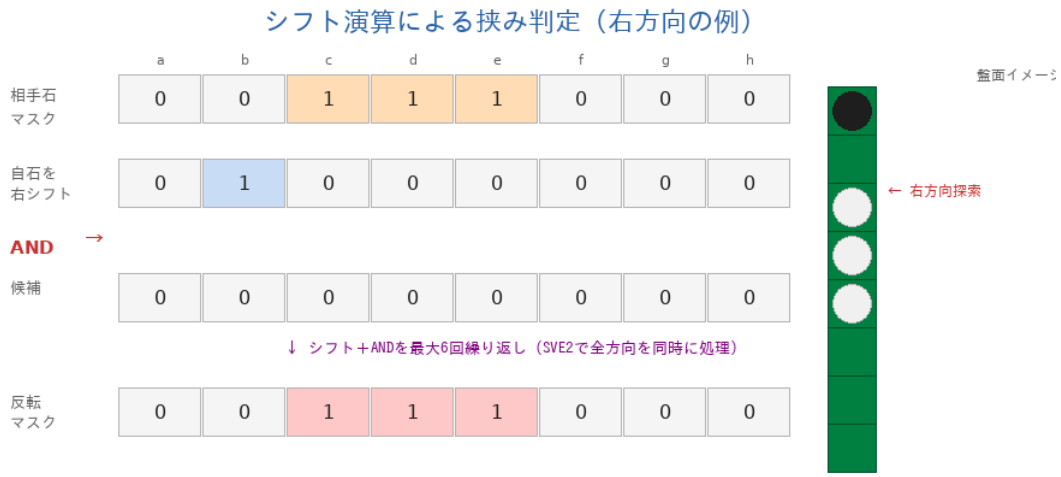


図 3: シフト＋AND 演算による挟み判定の流れ（右方向の例）

4.2 SVE2 命令の活用ポイント

SVE2 では、シフト量を各レーンで個別に指定できる可変シフト命令が利用可能です。これにより、8 方向で異なるシフト量を 1 命令で同時適用できます。

コード例: フリップ処理と NEON 盤面更新

全方向のフリップマスクを求めた後、NEON の veorq_u64 命令で黒白ビットボードを同時に XOR 反転します。

```
#include <arm_sve.h>
#include <arm_neon.h>

/*
 * flip_discs_sve2(my, opp, pos)
 *   概要 : 着手位置 pos に石を置いたとき、挟まれて
 *          裏返る相手石の位置をビットマスクで返す
 *   my   : uint64_t — 自石のビットボード
```

```
 *   opp  : uint64_t — 相手石のビットボード
 *   pos  : int      — 着手するマス番号 (0=左上, 63=右下)
 *   戻値 : uint64_t — 反転する相手石のビットマスク
 *            1 のビット位置 = 裏返すべき相手石の位置
 */
uint64_t flip_discs_sve2(uint64_t my, uint64_t opp,
                         int pos) {
    uint64_t flip_mask = 0;
    uint64_t placed    = 1ULL << pos;

    for (int dir = 0; dir < 8; dir++) {
        int      shift = SHIFT_AMOUNTS[dir];
        uint64_t mask  = EDGE_MASK[dir];
        uint64_t flip  = 0, cursor = placed;

        for (int i = 0; i < 6; i++) {
            /*
             * ビットシフト + 端マスク AND
             *   cursor : uint64_t — 現在のカーソル位置ビット
             *   shift  : int — 正値=左シフト / 負値=右シフト
             *   mask   : uint64_t — 端マスク(盤外回り込み防止)
             */
            cursor = (shift > 0)
                ? (cursor << shift) & mask
                : (cursor >> -shift) & mask;
            if (!(cursor & opp)) break;
            flip |= cursor;
        }
        uint64_t beyond = (shift > 0)
            ? (cursor << shift) & mask
            : (cursor >> -shift) & mask;
        if (beyond & my) flip_mask |= flip;
    }
    return flip_mask;
}

/*
 * apply_flip_neon(board, flip, placed)
 *   概要 : NEON の XOR 命令で黒白ビットボードを
 *          1 命令で同時更新し次局面を生成する
 *   board  : uint64x2_t — 現在の盤面 SIMD レジスタ
 *              レーン 0 = 黒ビットボード
 *              レーン 1 = 白ビットボード
 *   flip   : uint64_t   — 反転する相手石のビットマスク
 *   placed : uint64_t   — 着手した自石のビット(1ULL<<pos)
 *   戻値   : uint64x2_t — 更新後の盤面 SIMD レジスタ
 */
inline uint64x2_t apply_flip_neon(
        uint64x2_t board,
        uint64_t flip, uint64_t placed) {
    uint64_t data[2] = { flip | placed, flip };
    /*
     * vld1q_u64(ptr) / veorq_u64(a, b)
     *   黒レーン: flip|placed (自石追加+裏返し)
     *   白レーン: flip        (裏返された相手石を消す)
     *   ※ XOR 1 命令で黒白 2 ボードを同時更新
     */
    return veorq_u64(board, vld1q_u64(data));
}
```

リスト 3: フリップ処理と NEON XOR による盤面更新

◆ SIMD 命令 詳細解説

[1][2] cursor/beyond のシフト+AND ― C++の << / >> とビット論理 AND を組み合わせた処理。方向(正=左シフト/負=右シフト)に応じてカーソルを進め、端マスクで盤外を除去します。最大 6 回繰り返して挟み候補を蓄積します。

[3] ― 黒レーン用(flip|placed)と白レーン用(flip)の 2 値を 128 ビットレジスタに一括ロード。レーン別の反転マスクを 1 命令で準備します。

[4] ― NEON EOR 命令(Quadword/64bit lanes)。黒ビットボード・白ビットボードを 1 命令で同時に XOR 反転。XOR により反転マスクが 1 のビット位置の石が黒白入れ替わります。「石を置く＋挟んだ石を裏返す」が 1 命令で完結します。

◆ API 仕様 / 引数説明

関数名 (intrinsic)	戻り値型	主な引数	引数・動作の説明
vld1q_u64	uint64x2_t	const uint64_t *ptr	メモリから 64bit×2 要素(128bit)を 1 命令でロード。レーン用(flip|placed)と白レーン用(flip)を同時にベクル化する。
veorq_u64	uint64x2_t	uint64x2 t a, uint64x2_t b	a、b の各対応レーンをビット XOR。黒ビットボー白ビットボードを 1 命令で同時反転。石を置く+挟石を裏返す操作が 1 命令で完結する。
vst1q_u64	void	uint64_t *ptr, uint64x2_t val	128bit をメモリへ一括ストア。EOR 後の次局面ビボードを構造体へ書き出す。

5. 述語レジスタによる合法手フィルタリング

SVE2 の最大の特徴の 1 つが述語（プレディケート）レジスタです。これはベクトルの各レーンに対して「処理する／しない」を示すビットマスクで、条件分岐なしにデータの選択的処理を実現します。

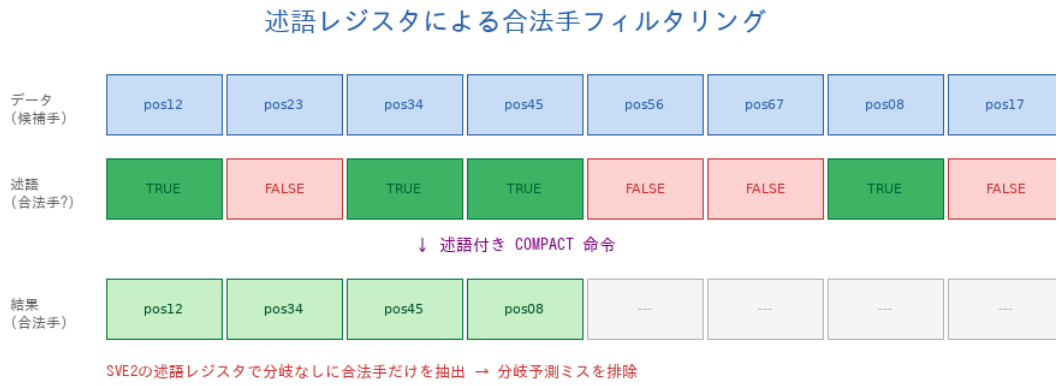


図 4: 述語レジスタによる合法手の選択的抽出

従来のスカラー処理では **if 文による分岐**で合法手を判定していました。分岐予測ミスによるパイプラインストールを、SVE2 の述語付き命令で**完全に排除**できます。

コード例: SVE2 COMPACT 命令による合法手抽出

WHILELT 命令で残り要素数に応じた述語を自動生成し、COMPACT 命令で合法手だけを詰め込みます。

```
#include <arm_sve.h>

/*
 * filter_legal_moves_sve2(candidates, legal_out,
 *                         count, legal_mask)
 *   概要 : SVE2 の述語レジスタと COMPACT 命令を使い、
 *          候補手の中から合法手だけを if 分岐なしで抽出する
 *   candidates : const uint8_t* — 候補手座標の配列
 *                  各要素は盤面マス番号(0〜63)
 *   legal_out  : uint8_t*       — 合法手座標の出力バッファ
 *                  呼び出し元が十分なサイズを確保すること
 *   count      : int            — candidates の要素数
 *   legal_mask : uint64_t       — 合法手ビットマスク
 *                  1 のビット位置 = 合法手のマス番号
 *   戻値       : int            — 抽出した合法手の個数
 */
int filter_legal_moves_sve2(
        const uint8_t* candidates,
        uint8_t*       legal_out,
        int            count,
        uint64_t       legal_mask) {
    int out_idx = 0, i = 0;
```

```
    while (i < count) {
        /*
         * svwhilelt_b8(base, limit)
         *   base : int64_t — 現在インデックス(i)
         *   limit: int64_t — 上限インデックス(count)
         *   戻値 : svbool_t — 残り要素数に応じた述語を生成
         */
        svbool_t pg = svwhilelt_b8(i, count);

        svuint8_t pos = svld1_u8(pg, &candidates[i]);

        svuint8_t bits = svlsr_x(pg,
            svreinterpret_u8(svdup_u64(legal_mask)), pos);

        svuint8_t is_legal =
            svand_x(pg, bits, svdup_u8(1));

        svbool_t pg_legal = svcmpne_u8(pg, is_legal, 0);

        /*
         * svcompact_u8(pg, v)     ← SVE2 COMPACT 命令
         *   pg  : svbool_t  — 抽出対象(合法手=TRUE)の述語
         *   v   : svuint8_t — 候補手座標ベクトル
         *   戻値: svuint8_t — TRUE レーンの要素を先頭に詰めた結果
         */
        svuint8_t compacted = svcompact_u8(pg_legal, pos);

        int n = svcntp_b8(pg_legal, pg_legal);
        svst1_u8(svwhilelt_b8(0, n),
                 &legal_out[out_idx], compacted);
        out_idx += n;
        i += svcntb();
    }
    return out_idx;
}
```

リスト 4: SVE2 述語レジスタと COMPACT 命令による合法手フィルタリング

◆ SIMD 命令 詳細解説

[1] ― インデックス i から count までの残り要素数に応じた述語を自動生成。ループ末尾の端数処理が不要になります。

[2] ― 述語が有効なレーンのみバイト単位でロード。境界外アクセスが発生しません。

[3] ― 可変シフト。各レーンで異なるシフト量(各候補座標の値)を使い、legal_mask の対応ビットを最下位へ引き出します。全候補の合法判定が 1 命令で完了します。

[4] ― 最下位ビットだけをマスクして取り出します。0 なら違法・1 なら合法の判定値になります。

[5] ― 0 でないレーン(合法手)を TRUE とする述語を生成。この述語が後続 COMPACT 命令の入力になります。

[6] ― **SVE2 COMPACT 命令。** pg が TRUE のレーンの要素だけをベクトルの先頭に詰めます。if 分岐なしにスカラーのフィルタループを完全代替する強力な命令です。

[7] ― 述語内の TRUE ビット数をカウント。今回処理した合法手の個数を求め、出力インデックスを正確に進めます。

[8] ― 述語付きバイトストア。有効レーンのみ出力バッファへ書き込みます。

[9] ― 現在の SVE2 実装のベクトル長(バイト数)を実行時に返します。DGX Spark では 16(128bit÷8)を返します。この VLA 設計で将来の広幅実装でも同一コードが動きます。

◆ API 仕様 / 引数説明

関数名 (intrinsic)	戻り値型	主な引数	引数・動作の説明
svwhilelt_b8	svbool_t	int64_t base, int64_t limit	base < limit の範囲内で 8bit レーンを TRUE にした述を返す。ループの残り要素数に応じた述語を自動生パディング不要にする。
svld1_u8	svuint8_t	svbool_t pg, const uint8_t *ptr	pg が TRUE のレーンのみ 1 バイトずつロード。境界アクセスなしに端数要素を安全に処理できる。
svlsr_x	svuint8_t	svbool t pg, svuint8 t op, svuint8_t shift	各レーン個別のシフト量(pos 値)で右シフト。可変シトにより全候補座標の合法判定を 1 命令で実行でき
svdup_u8	svuint8_t	uint8_t imm	8bit スカラー値を全レーンにブロードキャスト。マ値(1)の配布や比較基準値の準備に使用。
svand_x	svuint8_t	svbool_t pg, svuint8_t a, svuint8_t b	ビット AND。最下位ビットだけを取り出し合法(1)/(0)を判定する。
svcmpne_u8	svbool_t	svbool_t pg, svuint8_t a, uint8_t imm	各レーンで a≠imm を比較し述語を返す。0 との比較合法手レーンを TRUE にした述語を生成する。
svcompact_u8	svuint8_t	svbool_t pg, svuint8_t v	pg が TRUE のレーンの要素だけを先頭に詰める。if岐不要で合法手の前詰め処理を 1 命令で実現する S固有命令。
svcntp_b8	uint64_t	svbool_t pn, svbool_t pm	pm のうち pn が TRUE の位置の TRUE ビット数をカント。今回処理した合法手の個数を求め出力インデスを正確に進める。
svst1_u8	void	svbool_t pg, uint8_t *ptr, svuint8_t v	pg が TRUE のレーンのみ 1 バイトずつストア。合だけを出力バッファへ書き込む境界安全なストア命
svcntb	uint64_t	(引数なし)	現在の SVE2 実装のベクトル長(バイト数)を実行時す。128bit 実装では 16。VLA 設計でハードウェア存しないループ制御が可能。

6. 評価関数のベクトル化

探索中の各局面に対して「どちらが有利か」を数値で判定する評価関数は、思考ルーチンの知性の核です。典型的なオセロ評価関数では、盤面の各位置に重みを持つ評価テーブルを用い、特徴量との積和（内積）を計算します。

6.1 MLA 命令による積和演算

SVE2 の MLA（Multiply-Accumulate）命令は、乗算と加算を 1 命令で実行します。64 マスの評価をベクトル長ぶんずつ処理すれば、わずか数回の MLA 命令で完了します。

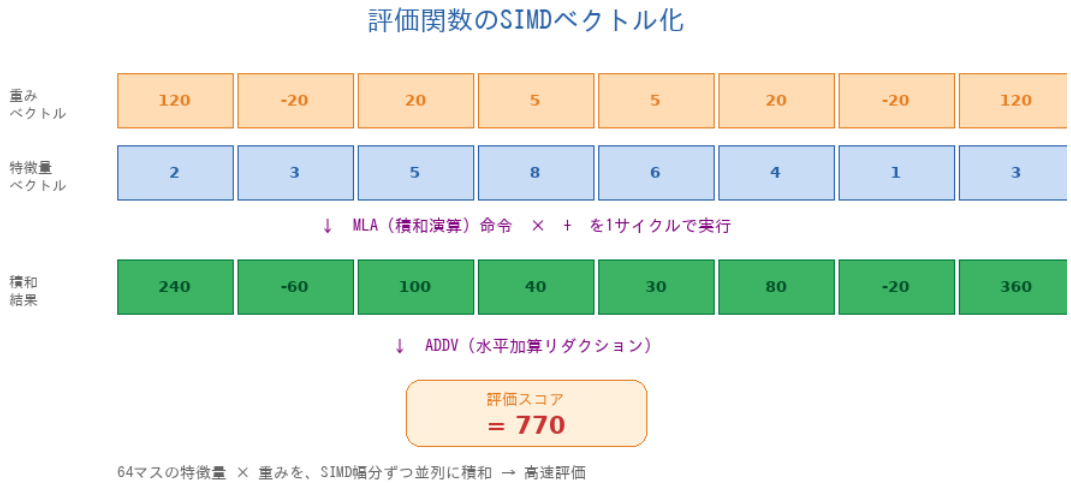


図 5: MLA 命令と ADDV リダクションによる評価関数の高速計算

6.2 ADDV 水平リダクション

MLA 命令で各レーンに部分和が求まった後、SVE2 の ADDV 命令でベクトル内の全要素を 1 命令で合算して最終スコアを得ます。

コード例: SVE2 MLA + ADDV による評価関数

64 マス分の重みと特徴量をベクトル長ずつロードし、svmla 命令で積和した結果を svaddv でリダクションします。

```
#include <arm_sve.h>

static const int16_t EVAL_WEIGHTS[64] = {
    120,-20, 20,  5,  5, 20,-20,120,
    -20,-40, -5, -5, -5, -5,-40,-20,
     20, -5, 15,  3,  3, 15, -5, 20,
      5, -5,  3,  3,  3,  3, -5,  5,
      5, -5,  3,  3,  3,  3, -5,  5,
     20, -5, 15,  3,  3, 15, -5, 20,
    -20,-40, -5, -5, -5, -5,-40,-20,
```

```
    120,-20, 20,  5,  5, 20,-20,120,
};

/*
 * evaluate_sve2(my, opp)
 *   概要 : SVE2 の MLA 命令(積和)と ADDV 命令(水平リダクション)で
 *          盤面の評価スコアを高速計算して返す
 *   my   : uint64_t — 自石のビットボード
 *            評価テーブルで重みに +1 を乗じるマス
 *   opp  : uint64_t — 相手石のビットボード
 *            評価テーブルで重みに -1 を乗じるマス
 *   戻値 : int32_t  — 盤面評価スコア
 *            正値 = 自石が有利 / 負値 = 相手が有利
 */
int32_t evaluate_sve2(uint64_t my, uint64_t opp) {
    svint16_t acc = svdup_s16(0);   // アキュムレータを 0 初期化

    int16_t features[64];
    for (int i = 0; i < 64; i++) {
        uint64_t bit = 1ULL << i;
        features[i] = (my&bit)?1 : (opp&bit)?-1 : 0;
    }

    for (int i = 0; i < 64; i += svcnth()) {
        svbool_t  pg = svwhilelt_b16(i, 64);
        svint16_t w  = svld1_s16(pg, &EVAL_WEIGHTS[i]);
        svint16_t f  = svld1_s16(pg, &features[i]);
        /*
         * svmla_s16_x(pg, acc, w, f)  ← MLA 命令
         *   pg  : svbool_t  — 処理対象レーンの述語
         *   acc : svint16_t — アキュムレータ(累積先)
         *   w   : svint16_t — 位置重み(EVAL_WEIGHTS)
         *   f   : svint16_t — 特徴量(+1 / 0 / -1)
         *   戻値: svint16_t — acc += w×f の結果
         */
        acc = svmla_s16_x(pg, acc, w, f);
    }

    /*
     * svaddv_s16(pg, v)  ← ADDV 命令 (水平リダクション)
     *   pg  : svbool_t  — 加算対象の述語(全レーン)
     *   v   : svint16_t — 各レーンの部分積和(acc)
     *   戻値: int16_t   — 全レーンの総和スカラー
     */
    int16_t score = svaddv_s16(svptrue_b16(), acc);
    return (int32_t)score;
}
```

リスト 5: SVE2 MLA 積和 + ADDV 水平加算による評価関数

◆ SIMD 命令 詳細解説

[1] — 全レーンを符号付き 16bit 値 0 で初期化したベクトルを生成。MLA 命令の累積アキュムレータをゼロクリアします。

[2] — 現在実装の SVE2 ベクトル長をハーフワード(16bit)数で返します。128bit 実装では 8 を返し、ループステップ幅として使います。

[3] — 残りマス数に応じた述語を生成。64 が 16 の倍数でない場合も端数を自動処理しパディング不要です。

[4] ― 述語が有効なレーンのみ符号付き 16bit をロード。重みテーブルと特徴量を各 1 命令でベクトルへ取り込みます。

[5] ― **SVE2 MLA 命令(Multiply-Accumulate)。** acc += w × f を乗算+加算の 1 命令で実行。スカラーなら 1 マス 2 命令が必要なところをベクトル長分(128bit 時 8 要素)まとめて処理します。

[6] ― **SVE2 ADDV 命令(Add Across Vector)。** ベクトル全レーンを 1 命令で合算しスカラーを返す水平リダクション。スカラーでは要素数-1 回の加算が必要な処理が 1 命令で完了します。

◆ API 仕様 / 引数説明

関数名 (intrinsic)	戻り値型	主な引数	引数・動作の説明
svdup_s16	svint16_t	int16_t imm	符号付き 16bit 値を全レーンに複製。MLA 命令のアムレータ初期化(0 クリア)に使用する。
svcnth	uint64_t	(引数なし)	現在実装の SVE2 ベクトル長をハーフワード(16bit)返す。128bit 実装では 8。ループステップ幅の決定用。
svwhilelt_b16	svbool_t	int64_t base, int64_t limit	base < limit の範囲で 16bit レーンを TRUE にした述を返す。64 マスの端数処理を自動化し境界チェック排除する。
svld1_s16	svint16_t	svbool_t pg, const int16_t *ptr	pg が TRUE のレーンのみ符号付き 16bit をロード。みテーブル・特徴量を各 1 命令でベクトルに取り込
svmla_s16_x	svint16_t	svbool_t pg, svint16_t acc, svint16_t w, svint16_t f	acc += w × f を乗算+加算の 1 命令で実行する MLA(Multiply-Accumulate)命令。128bit 時に 8 要素サイクルで処理。_x は inactive レーン未定義=最速
svaddv_s16	int16_t	svbool_t pg, svint16_t v	pg が有効な全レーンを合算しスカラーを返す水平リクション(Add Across Vector)。要素数-1 回の加算が命令で完了する。
svptrue_b16	svbool_t	(引数なし)	全 16bit レーンを TRUE にした述語を返す。svaddv_s16 に全レーンを対象として渡す際に使用る。

7. ポピュレーションカウント（石数計算）

ビットボードで「1 が立っているビットの数」をカウントするポピュレーションカウント（popcount）は、石の数や着手可能数の計算に不可欠です。



図 6: CNT 命令によるビットカウントとその応用

SVE2/NEON の CNT 命令は、ビットボード内の 1 のビット数を高速に数えます。終局時の勝敗判定、mobility 評価、パターンマッチングなど、あらゆる場面で使用されます。

コード例: NEON/SVE2 CNT 命令による石数カウント

NEON 版では vcnt_u8 でバイト単位のビットカウント後に vaddlv_u8 で総和します。SVE2 版では svcnt + svaddv で同様の処理を行います。

```
#include <arm_neon.h>
#include <arm_sve.h>

/*
 * popcount_neon(bits)
 *   概要 : NEON CNT 命令で 64bit ビットボードの
 *          1 ビット数(立っているビットの個数)を高速カウントする
 *   bits : uint64_t — カウント対象のビットボード
 *   戻値 : int      — 1 ビットの個数 (0〜64)
 */
inline int popcount_neon(uint64_t bits) {
    uint8x8_t v = vreinterpret_u8_u64(
                      vcreate_u64(bits));
    /*
     * vcnt_u8(a) ← CNT 命令
     *   a    : uint8x8_t — 8 バイトの入力ベクトル
     *   戻値 : uint8x8_t — 各バイトのビット数を並列カウント
     */
    uint8x8_t cnt = vcnt_u8(v);
    /*
     * vaddlv_u8(a) ← ADDLV 命令
     *   a    : uint8x8_t — CNT 結果ベクトル
     *   戻値 : uint16_t  — 全 8 バイトのビット数の総和
     */
    return (int)vaddlv_u8(cnt);
}
```

```
/*
 * popcount_sve2(bits)
 *   概要 : SVE2 CNT+ADDV 命令で popcount_neon と同等の処理を
 *          可変長ベクトル対応で実装した SVE2 版
 *   bits : uint64_t — カウント対象のビットボード
 *   戻値 : int      — 1 ビットの個数 (0〜64)
 */
inline int popcount_sve2(uint64_t bits) {
    svbool_t  pg = svptrue_b8();
    svuint8_t v  = svld1_u8(pg,
        reinterpret_cast<const uint8_t*>(&bits));
    svuint8_t cnt = svcnt_u8_x(pg, v);
    return (int)svaddv_u8(pg, cnt);
}

/*
 * mobility(legal_mask)
 *   概要 : 合法手ビットマスクの bitcount で
 *          現局面の着手可能数(mobility)を返す
 *   legal_mask : uint64_t — 合法手ビットマスク
 *   戻値       : int      — 着手可能手数 (0〜64)
 */
inline int mobility(uint64_t legal_mask) {
    return popcount_neon(legal_mask);
}

struct GameResult { int black, white, winner; };

/*
 * judge_game(b)
 *   概要 : 終局後の盤面から黒白の石数を数え
 *          勝敗を判定して返す
 *   b      : const Board& — 終局盤面
 *   戻値   : GameResult
 *     .black  : int — 黒石の個数
 *     .white  : int — 白石の個数
 *     .winner : int — 1=黒勝ち / -1=白勝ち / 0=引き分け
 */
GameResult judge_game(const Board& b) {
    GameResult r;
    r.black  = popcount_neon(b.black);
    r.white  = popcount_neon(b.white);
    r.winner = (r.black > r.white) ?  1 :
               (r.white > r.black) ? -1 : 0;
    return r;
}
```

リスト 6: NEON/SVE2 による popcount 実装と終局判定

◆ SIMD 命令 詳細解説

[1] ― NEON CNT 命令。8 バイトのベクトル各バイトについて立っているビット数を並列カウント。64bit を 8 バイトに分割して処理することで 8 バイト同時カウントを実現します。

[2] ― NEON ADDLV 命令(Add Long across Vector)。8 つの 8bit 値を 16bit に拡張しながら総和。CNT 結果を合算して popcount 最終値を得ます。

[3] ― SVE2 版 CNT 命令。各バイトのビット数を述語付きで並列カウント。可変長に対応し将来の広幅実装でも同一コードで動作します。

[4] ― SVE2 ADDV。全バイトのカウントを 1 命令で総和してスカラー返却。NEON 版 vaddlv_u8 に相当し述語で有効範囲を制御できます。

[5][6] popcount_neon 応用 ― mobility 計算には合法手マスクの bitcount、終局判定には黒・白ビットボードの bitcount をそれぞれ 1 関数呼び出しで即座に取得します。

◆ API 仕様 / 引数説明

関数名 (intrinsic)	戻り値型	主な引数	引数・動作の説明
vcreate_u64	uint64x1_t	uint64_t a	スカラーuint64_t を 64bit 幅の NEON ベクトル型に換。vreinterpret 系との組み合わせで型変換チェーン起点となる。
vreinterpret_u8_u64	uint8x8_t	uint64x1_t a	64bit 値を 8×8bit として再解釈(ビット変換なし)。vcnt_u8 が 8bit 粒度を要求するための型キャスト。
vcnt_u8	uint8x8_t	uint8x8_t a	8 バイトの各バイトについてビット数を並列カウンる CNT 命令。64bit を 8 バイトに分割することで 8 ト同時カウントを実現。
vaddlv_u8	uint16_t	uint8x8_t a	8 つの 8bit 値を 16bit に拡張しながら横加算する ADDLV 命令。vcnt_u8 の結果を合算して popcount 値を得る。
svcnt_u8_x	svuint8_t	svbool_t pg, svuint8_t v	SVE2 版 CNT 命令。各バイトのビット数を述語付き並列カウント。可変長ベクトルに対応し将来の広幅でも同一コードで動作する。
svaddv_u8	uint8_t	svbool_t pg, svuint8_t v	SVE2 ADDV。全バイトのカウントを 1 命令で総和スカラーを返す水平リダクション。述語で有効範囲御できる。
svld1_u8	svuint8_t	svbool_t pg, const uint8_t *ptr	述語が有効なレーンのみバイト単位でロード。64bi ットボードを 8 バイト単位で SVE2 ベクトルとして り込む。

8. SIMD 活用による性能向上

各処理で SIMD を活用した場合の高速化効果のイメージを以下に示します。

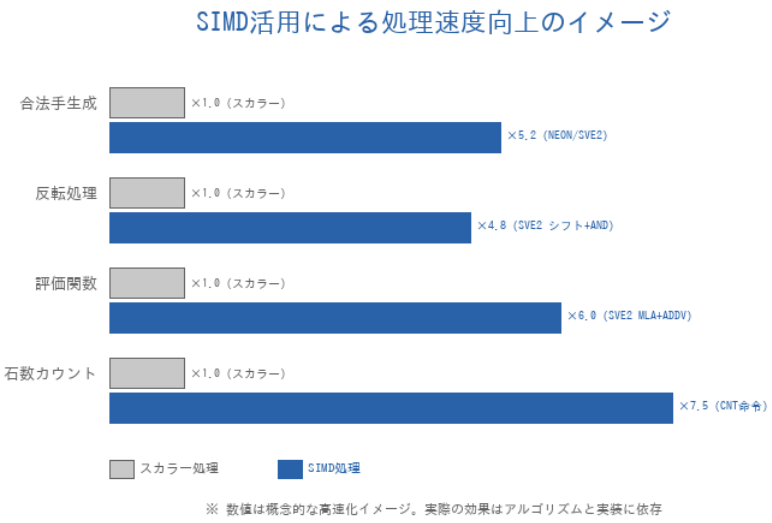


図 7: 各処理における SIMD 高速化のイメージ（概念図）

8.1 主要処理と対応 SIMD 命令

処理	使用する SIMD 命令	効果
合法手生成	SHL/SHR, AND, ORR (×8 レーン)	8 方向同時探索
反転処理	可変シフト, AND, BIC	全方向同時フリップ
評価関数	MLA, ADDV	積和＋リダクション一括
石数カウント	CNT	popcount 1 命令
合法手選択	COMPACT + 述語	分岐レス抽出
着手可能数	CNT + ADDV	mobility 高速計算

8.2 DGX Spark の実行スループット

DGX Spark の Cortex-X925（P コア）は 128 ビットの NEON/SVE2 命令を 1 サイクルに最大 6 個リタイアできます。Cortex-A725（E コア）でも 2 個リタイア可能です。

9. DGX Spark 上のオセロ AI 全体像

DGX Spark の最大の特徴は、CPU（ARM SIMD）と GPU（Blackwell）が 128GB の統一メモリを共有する UMA アーキテクチャです。CPU 側の SIMD 処理による高速な盤面操作・探索と、GPU 側の Tensor Core によるニューラルネットワーク評価を、メモリコピーなしで組み合わせることが可能です。

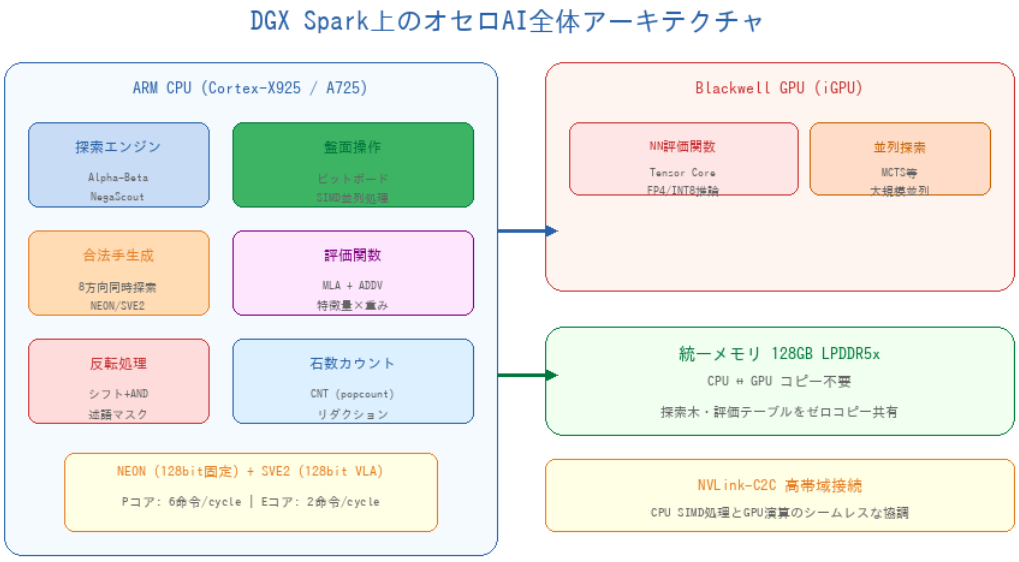


図 8: DGX Spark 上のオセロ AI 全体アーキテクチャ

コード例: SIMD 高速探索エンジン

ここまで解説した各 SIMD 関数を組み合わせた Alpha-Beta 探索の全体像です。

```
#include <arm_neon.h>

/*
 * alpha_beta(board, depth, alpha, beta, my_turn)
 *   概要 : SIMD 命令で高速化した Alpha-Beta 木探索。
 *          合法手生成・フリップ・評価をすべて SIMD 関数で処理する
 *   board   : Board — 探索開始局面の盤面
 *   depth   : int   — 残り探索深さ (0 になったら評価関数を呼ぶ)
 *   alpha   : int   — Alpha カット値 (現在の自分側の下限スコア)
 *   beta    : int   — Beta カット値 (現在の相手側の上限スコア)
 *   my_turn : bool  — true=自石手番 / false=相手石手番
 *   戻値    : int   — この局面の最善スコア
 *               正値 = 自石有利 / 負値 = 相手有利
 */
int alpha_beta(Board board, int depth,
               int alpha, int beta, bool my_turn) {

    // 深さ 0: SVE2 評価関数で局面スコアを計算して返す
    if (depth == 0)
        return evaluate_sve2(board.black, board.white);

    uint64_t my  = my_turn ? board.black : board.white;
```

```
    uint64_t opp = my_turn ? board.white : board.black;

    // SVE2 並列合法手生成
    uint64_t legal =
        generate_legal_moves_sve2(my, opp);

    if (legal == 0)    // 合法手なし → パス
        return -alpha_beta(board, depth-1,
                           -beta, -alpha, !my_turn);

    int best = -999999;
    while (legal) {
        /*
         * __builtin_ctzll(x)
         *   x    : unsigned long long — 合法手ビットマスク
         *   戻値 : int — 最下位 1 ビットの位置(着手マス番号)
         */
        int pos = __builtin_ctzll(legal);
        legal &= legal - 1;

        // flip_discs_sve2(): 裏返す石のビットマスクを取得
        uint64_t flip   = flip_discs_sve2(my, opp, pos);
        uint64_t placed = 1ULL << pos;

        uint64_t fdata[2] = {
            my_turn ? flip | placed : flip,
            my_turn ? flip          : flip | placed };

        /*
         * vld1q_u64 / veorq_u64 / vst1q_u64
         *   現局面(board)を 128bit にロード → XOR で次局面生成
         *   → next 構造体に 1 命令でストア
         *   3 命令だけで次局面の盤面生成が完結する
         */
        uint64x2_t bw = vld1q_u64((uint64_t*)&board);
        bw = veorq_u64(bw, vld1q_u64(fdata));
        Board next;
        vst1q_u64((uint64_t*)&next, bw);

        int val = -alpha_beta(next, depth-1,
                              -beta, -alpha, !my_turn);
        if (val > best) best = val;
        if (val > alpha) alpha = val;
        if (alpha >= beta) break;   // β 枝刈り
    }
    return best;
}
```

リスト 7: SIMD 活用 Alpha-Beta 探索エンジン

◆ SIMD 命令 詳細解説

[1] ― 葉ノードで第 6 章の SVE2 MLA+ADDV 評価関数を呼び出します。探索末端で大量に呼ばれるため、この関数の SIMD 化が全体性能に最も貢献します。

[2] ― 第 3 章の 8 方向 SIMD 並列合法手生成。探索中に何百万回も呼ばれる最頻出処理です。

[3] ― GCC/Clang 組み込み関数「Count Trailing Zeros(64bit)」。合法手マスクの最下位ビット位置(候補手座標)を 1 命令で取得します。ARM 命令の RBIT+CLZ に変換されます。

[4] ― 第 4 章の SIMD フリップ処理。各候補手に対してフリップマスクを計算します。

[5] ― 現在の盤面構造体を 128 ビットレジスタへ一括ロード。黒白 2 ビットボードを 1 命令で取り込みます。

[6] ― NEON EOR 命令で現局面とフリップマスクの XOR を取り次局面を 1 命令で生成。黒白 2 ボードを同時処理します。

[7] ― 128 ビット一括ストアで次局面を構造体に書き出します。LD1→EOR→ST1 の 3 命令で次局面生成が完結します。

◆ API 仕様 / 引数説明

関数名 (intrinsic)	戻り値型	主な引数	引数・動作の説明
__builtin_ctzll	int	unsigned long long x	GCC/Clang 組み込み関数。x の最下位から連続するビット数(Count Trailing Zeros)を返す。ARM 命令の RBIT+CLZ に変換され 1 命令相当で実行。合法手マから候補座標を取り出すのに使用。
vld1q_u64	uint64x2_t	const uint64_t *ptr	現在の盤面構造体を 128bit レジスタへ一括ロード。白 2 ビットボードを 1 命令で取り込み EOR 命令へ力とする。
veorq_u64	uint64x2_t	uint64x2_t a, uint64x2_t b	128bit XOR。現局面とフリップマスクの XOR を取局面を 1 命令で生成。黒白 2 ボードを同時処理し 2 の個別演算を 1 回に削減する。
vst1q_u64	void	uint64 t *ptr, uint64x2_t val	128bit 一括ストアで次局面を構造体に書き出す。LD1→EOR→ST1 の 3 命令だけで次局面生成が完結る。

10. まとめ

本資料では、NVIDIA DGX Spark の ARM CPU に搭載された NEON/SVE2 SIMD 命令が、オセロ思考ルーチンの各処理段階でどのように活用できるかを、C++サンプルコード・SIMD 命令解説・API 仕様とともに解説しました。

活用領域	SIMD 技法	主な利点
ビットボード表現	128bit レジスタに黒白格納	盤面全体を 1 レジスタで操作
合法手生成	8 レーン並列シフト＋AND	8 方向を 1 命令で同時探索
反転処理	可変シフト＋マスク AND 連鎖	最頻出処理を大幅高速化
述語フィルタリング	述語レジスタ＋COMPACT	分岐予測ミスを完全排除
評価関数	MLA 積和＋ADDV リダクション	内積計算を数命令に圧縮
石数カウント	CNT ポピュレーションカウント	1 命令で bitcount 完了

DGX Spark の Cortex-X925 は 1 サイクルに 6 つの SIMD 命令をリタイアできる実行幅を持ち、加えて 128GB の統一メモリによる CPU-GPU 間のゼロコピーデータ共有が可能です。

今後、SVE2 の述語駆動型プログラミングと GPU の Tensor Core を組み合わせた**ハイブリッド AI 探索**が、オセロに限らず様々なボードゲーム AI の新しい標準となることが期待されます。

本資料に掲載した C++コードは arm_neon.h および arm_sve.h の intrinsic を用いた概念実装です。DGX Spark 上でのコンパイルには、GCC/Clang で -march=armv8-a+sve2 フラグを指定してください。